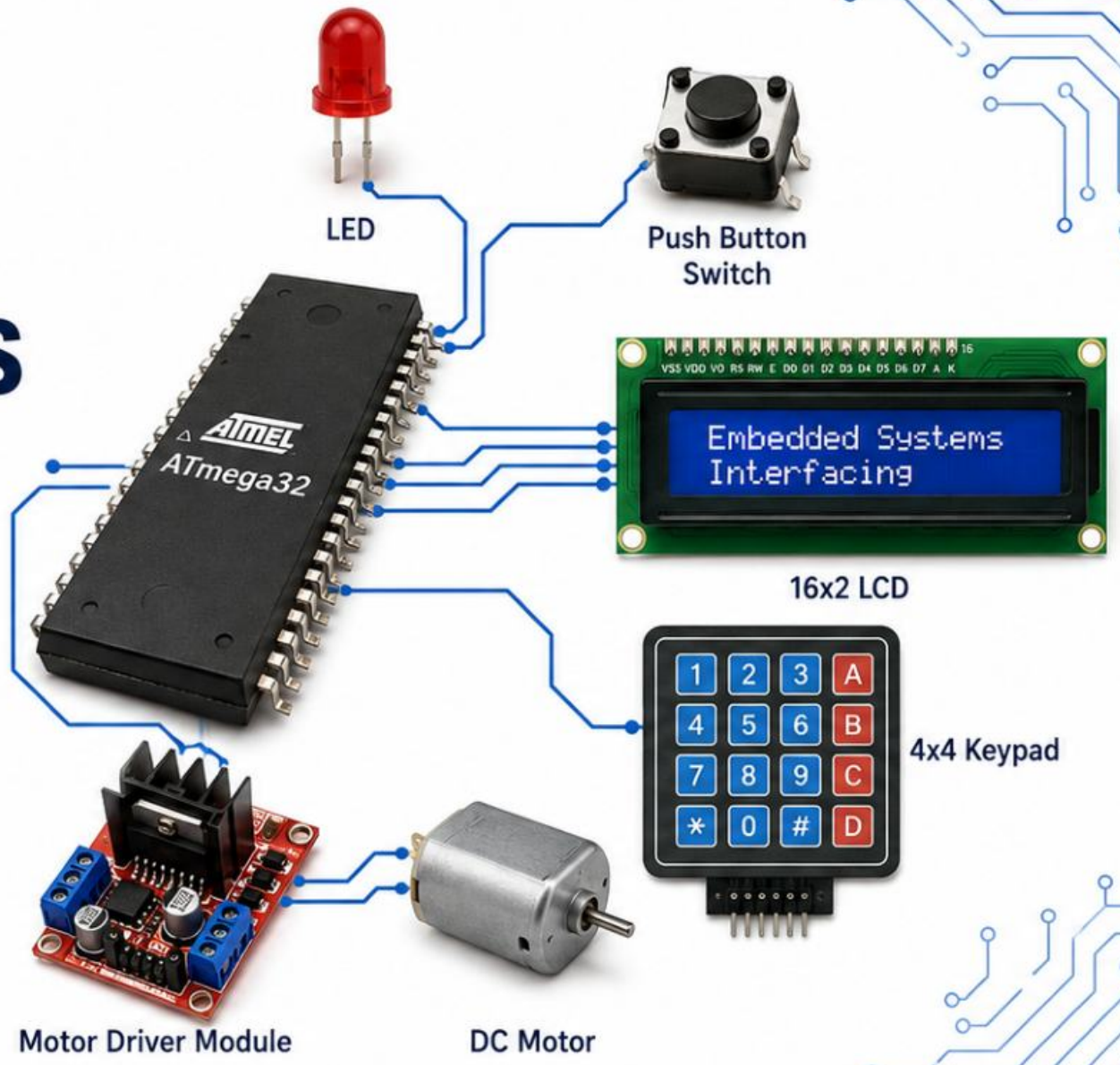
NTI | National Telecommunication Institute

Embedded Systems Interfacing

Digital I/O, Pins, Buffers, and Practical Device Interfaces

Prepared by Mahmoud Morsy

NTI | National Telecommunication Institute



NTI | National Telecommunication Institute

Session Roadmap

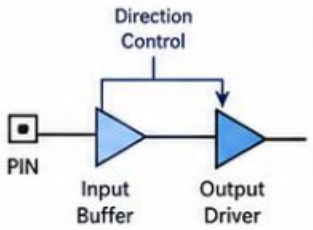
From digital pins to practical embedded-system interfaces

1 Digital I/O Fundamentals

HIGH, LOW, input, output

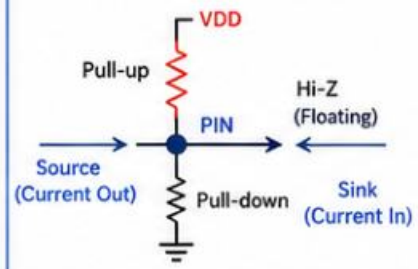
2 MCU Pins and Buffers

direction control, input buffer, output driver



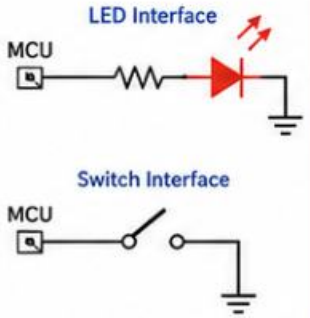
3 Source, Sink, and Floating Pins

current path, pull-up, pull-down, Hi-Z



4 Basic Interfaces

LED and Switch

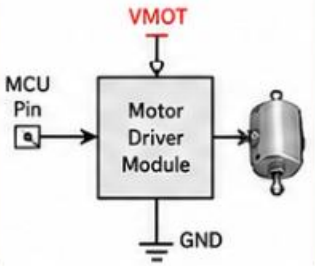


5 Character LCD and Keypad

datasheet-based interface, timing, scanning

6 Motor and Driver Interface

why a driver is needed, protection, current handling



By the end of this session, students will be able to:

- **Read a datasheet** to understand pin functions, ratings, and timing.
- **Design a safe interface circuit** using good practices, protection, and proper current handling.
- **Choose the correct MCU pins and driver** for reliable and efficient embedded-system designs.

NTI | National Telecommunication Institute

Digital I/O Fundamentals

Digital I/O (Input/Output) allows the ATmega32 to communicate with the outside world using logic levels.

LOGIC LEVELS (ATmega32)

V_{CC} — HIGH = V_{CC} — Logic 1

0 V — LOW = 0 V — Logic 0

DIGITAL INPUT

Input pins sense external logic levels (HIGH or LOW) and the MCU reads the state.

Example: reading a push button (Switch).

DIGITAL OUTPUT

Output pins drive external loads by setting HIGH or LOW.

Example: driving an LED.

I/O REGISTERS (Per Port x = B, C, D)

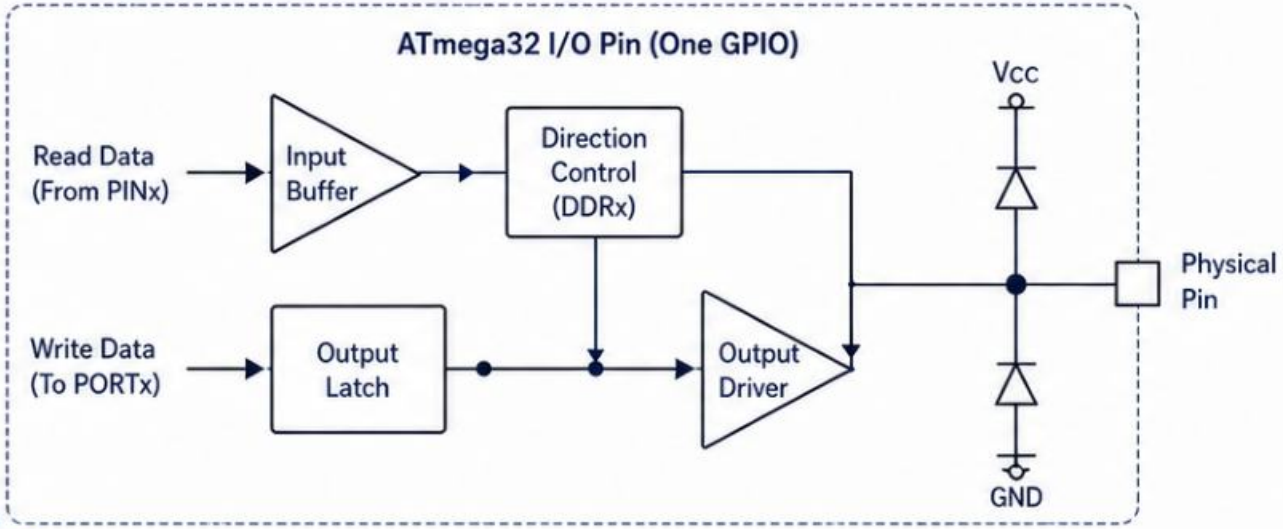
DDRx	Data Direction Register	Selects direction of each pin: 0 = Input, 1 = Output
PORTx	Port Data Register	Writes output value when DDRx = 1; Enables internal pull-up when DDRx = 0
PINx	Pin Input Register	Reads the current logic state of the pin

I/O BEHAVIOR & EXAMPLES

DDRx Bit	Pin Direction	Action / Meaning	Example
0	Input	Pin is high-impedance input: Reads external logic level.	Read a switch: Switch pressed → reads 0 (LOW) Switch released → reads 1 (HIGH) (using pull-up)
1	Output	Pin drives output according to PORTx bit.	Drive an LED: PORTx = 1 → LED ON (HIGH) PORTx = 0 → LED OFF (LOW)

MCU Pins, Buffers, and Pin States

Inside an ATmega32 Digital I/O Pin



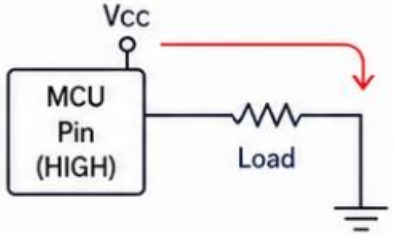
Note: Internal Pull-up and Floating Inputs on AVR

- Enable the internal pull-up by setting PORTx bit HIGH while the pin is configured as input (DDRx bit = 0).
- Floating inputs pick up electrical noise and can randomly toggle between HIGH and LOW, leading to unstable or inconsistent readings.

Pin States and Current Flow

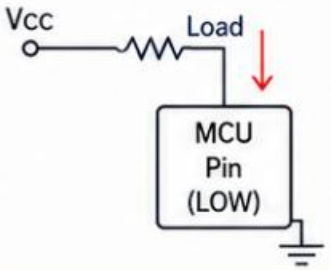
1 Source Current (HIGH)

Pin outputs HIGH (≈ Vcc). Current flows from MCU pin to load to ground.



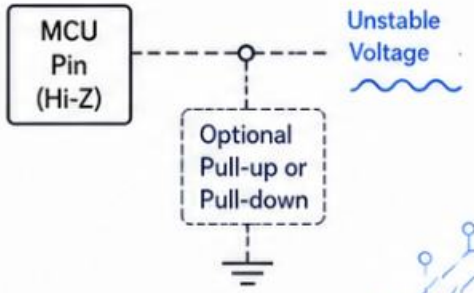
2 Sink Current (LOW)

Pin outputs LOW (≈ 0V). Current flows from Vcc through load into MCU pin to ground.



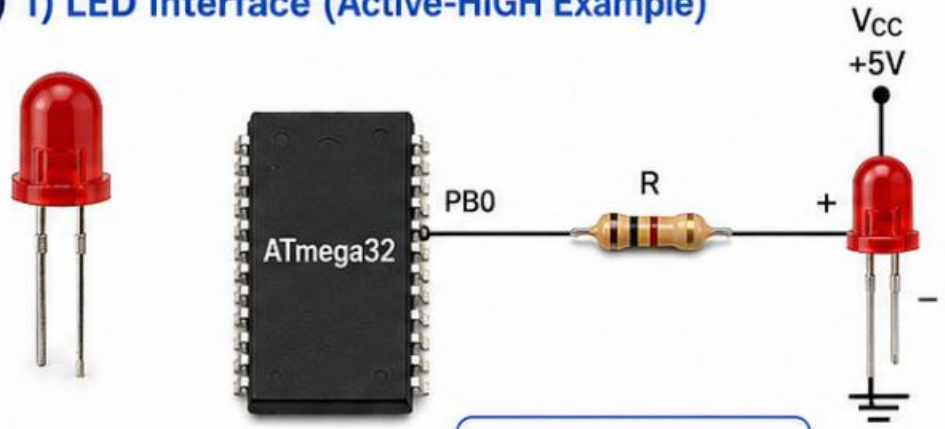
3 Floating Pin / Hi-Z (Input)

Pin not driven (Hi-Z). Voltage is undefined and easily affected by noise.
Use pull-up or pull-down to make the state stable.



Interfacing an LED and a Switch

1) LED Interface (Active-HIGH Example)



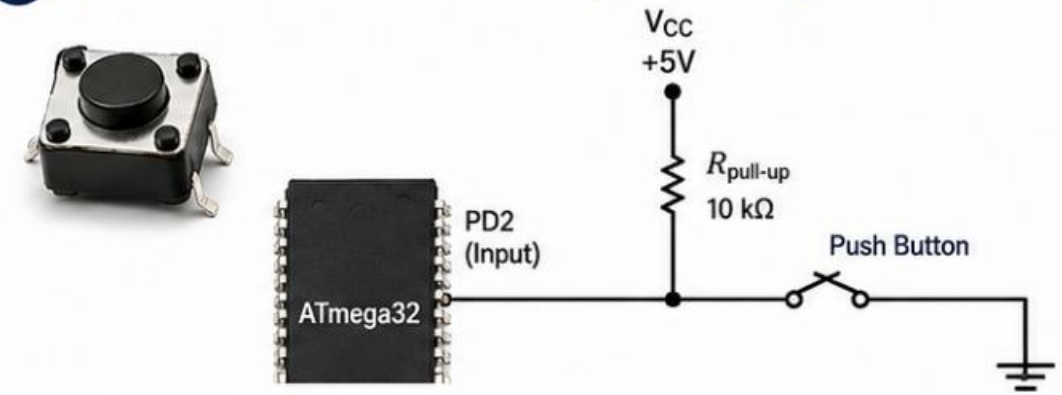
$$R = \frac{V_{CC} - V_{LED}}{I_{LED}}$$

- Resistor limits current to protect both the LED and the MCU pin.

Operation (Active-HIGH):

- PB0 = HIGH (≈ Vcc) → current flows → LED turns ON
- PB0 = LOW (0V) → no current → LED turns OFF

2) Push-Button Switch Interface (Active-LOW)



How it works (Active-LOW):

- Button NOT pressed → input pulled HIGH by $R_{pull-up}$
- Button pressed → pin connected to GND → reads LOW

Why pull-up is needed:

- Without pull-up, the input is floating when the button is not pressed, leading to random (unreliable) readings.

Debouncing:

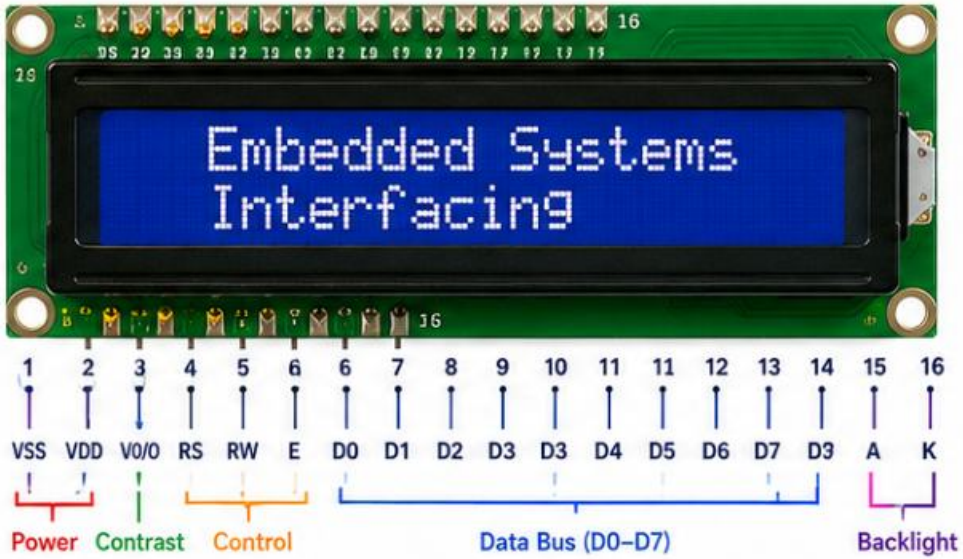
- Mechanical buttons bounce for a few ms.
- Use a small delay (e.g., 10–20 ms) or software debounce.

Practical Example:
Press button → MCU reads LOW → LED turns ON

NTI National Telecommunication Institute

Character LCD Interface (16x2 LCD)

The **16x2** character LCD (HD44780 compatible) displays **32 characters** (16 columns × 2 rows). It uses a parallel interface with control and data lines.



Power Pins
VSS (GND) and VDD (+5V) power the LCD.

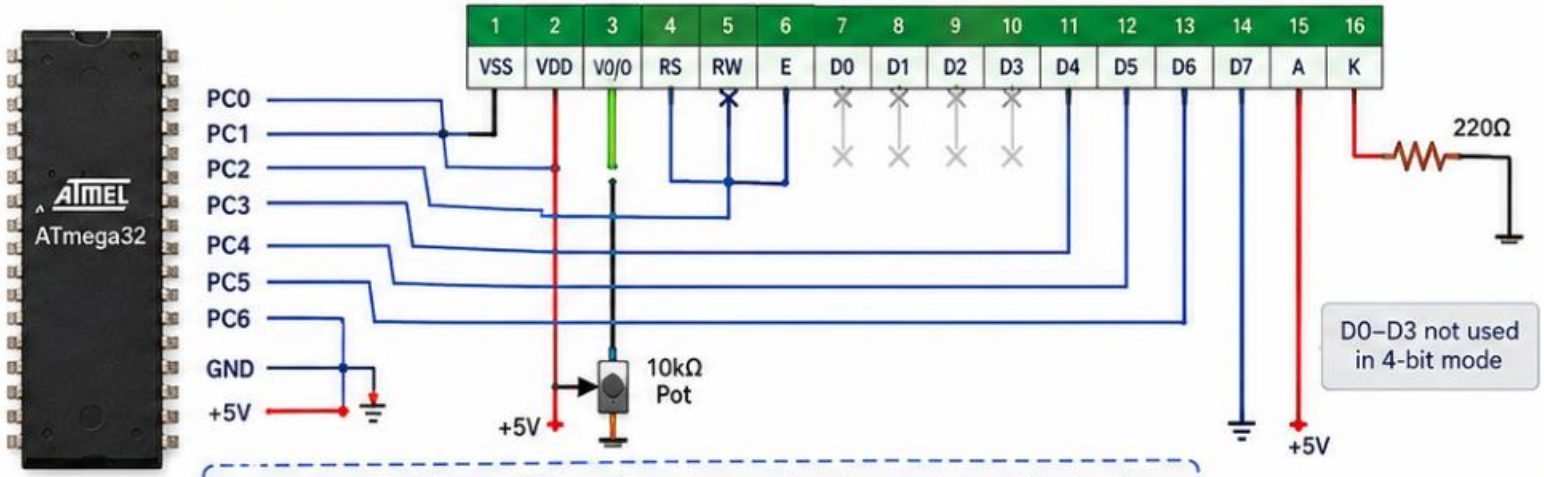
Contrast Pin
V0/VO sets the display contrast. Use a potentiometer between 5V and GND.

Control Pins
RS selects instruction (0) or data (1).
RW selects read (1) or write (0).
E enables data/instruction on rising edge.

Data Pins
D0–D7 are the 8-bit data bus.
In 4-bit mode, only D4–D7 are used.

Backlight Pins
A is LED anode (+5V) and K is LED cathode (GND) for the backlight.

Wiring Example to ATmega32 (4-bit Mode)



Note: Tie RW to GND for write-only operation (common in applications). Use a 10kΩ potentiometer on V0/VO to adjust the display contrast.

NTI National Telecommunication Institute

LCD Operation from the Datasheet

16x2 Character LCD (HD44780 Compatible)

Control Signals

RS (Register Select)

RS = 0: Instruction register
RS = 1: Data register

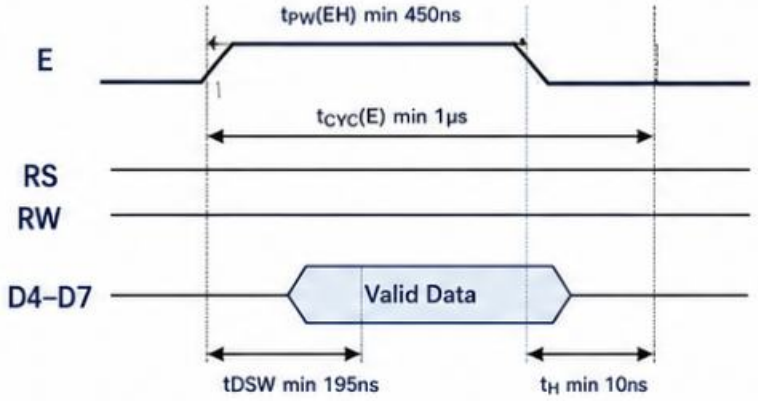
RW (Read/Write)

RW = 0: Write to LCD
RW = 1: Read from LCD

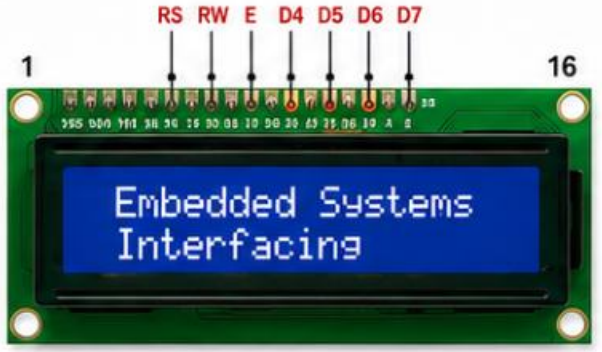
E (Enable)

A high-to-low pulse latches data on the rising edge and executes on the falling edge.

Enable (E) Timing (Write Cycle)



Initialization Sequence (4-bit Mode)



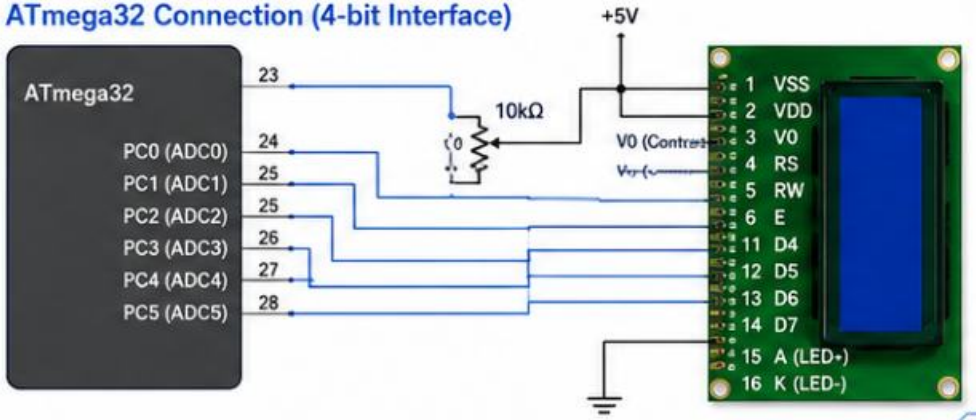
16x2 LCD

- HD44780 compatible
- 5V Operation
- 16 Columns x 2 Rows

Common Commands

Command	Hex	Description
Function Set	0x28	4-bit mode, 2-line, 5x8 font
Display ON/OFF	0x0C	Display ON, Cursor OFF, Blink OFF
Entry Mode Set	0x06	Increment cursor, No shift
Clear Display	0x01	Clear display and return cursor home
Return Home	0x02	Return cursor to home position

ATmega32 Connection (4-bit Interface)



NTI | National Telecommunication Institute

4x4 Keypad Interface

What is a 4x4 Keypad?

- A 4x4 membrane keypad has 16 keys arranged in 4 rows and 4 columns.
- Keys are normally open (NO). When a key is pressed, it connects one row line to one column line.

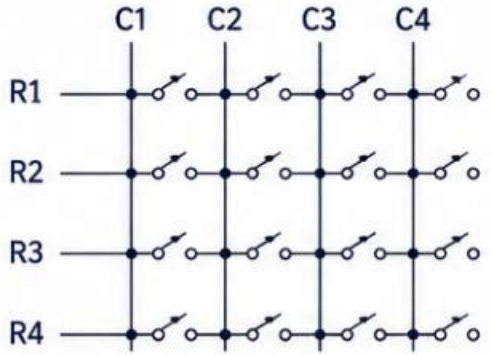
Matrix Arrangement

- Arranged as a matrix to reduce the number of MCU pins.
- Only 8 MCU I/O pins are needed instead of 16.
- Scanning is done by driving rows or columns sequentially and reading the other side to detect a key press.

Practical Considerations

- Use internal pull-up resistors on the input side.
- Debounce in software (10–20 ms typical).

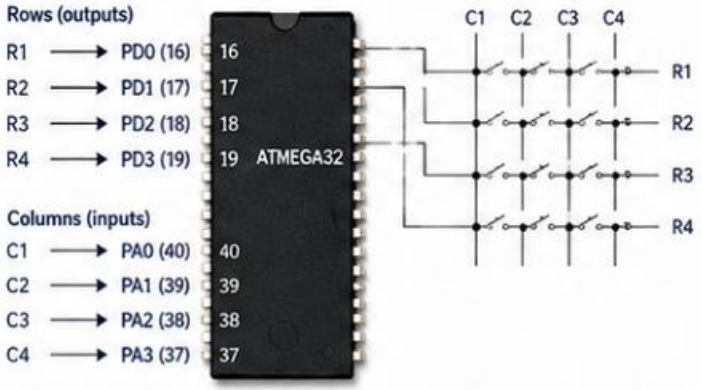
Matrix Diagram



Key Map

R \ C	C1	C2	C3	C4
R1	1	2	3	A
R2	4	5	6	B
R3	7	8	9	C
R4	*	0	#	D

Wiring Example to ATmega32



Scanning Sequence (Row Drive Example)

1. Set R1 = LOW, R2–R4 = HIGH.
2. Read C1–C4. If any column is LOW → key in R1 is pressed.
3. Repeat for R2, R3, R4.
4. Identify key from row and column.

Notes

- Enable internal pull-ups on column pins (PA0–PA3).
- Ensure only one row is LOW at a time.
- Software debounce: wait 10–20 ms after a key is detected and confirm.